

ORBIT: A Python-Based Interactive Simulation Game for Exploring Orbital Motion

By GC Zyanathanjho S. Labonete and Will Tyrone B. Araneta · CS5 Capstone Project Proposal

Introduction

1.1 Background of the Problem

Orbital motion is an important concept in physics involving gravity, velocity, distance and movement through foreign objects. However, understanding these factors interacting with each other can be difficult if presented through equations and theoretical explanations. Though students may understand the formula, they have limited yet no control with these variables and can't really observe the effects of an object motion if the variable is set to change.

The main difficulty is the lack of control. In a textbook or lecture the variables are fixed, so students cannot change a value and watch the consequences unfold. They cannot see how a slightly lower launch speed turns a circular orbit into an ellipse, or how an object falls inward when it moves too slowly.

Interactive simulations close this gap by letting learners change variables and observe the results. ORBIT is a Python-based simulation game that does this offline on ordinary school computers.

1.2 Statement of the Problem

This project asks one general question: how can an interactive python simulation game help students understand how mass, velocity, and distance determine orbital motion?

It answers four specific questions:

1. How accurately can ORBIT model orbital motion, measured against known results such as circular-orbit speed and Kepler's laws?
2. What data structures and algorithms are used to keep the simulation fast as the number of bodies grows, and where does measured performance differ from theoretical complexity?
3. How can the game let players change variables, predict the outcome, and then check their prediction against the simulation?
4. How do students' answers to orbit-prediction questions compare before and after playing ORBIT?

1.3 Objectives

Our main objective is to develop our own game named ORBIT, which is a python-based interactive game that allows users to explore orbital motion in action by manipulating variables and observing resulting trajectories based on the user's input.

The project aims to:

1. Develop a model that simulates an object's motion under gravitational influence
2. Implement algorithms that calculate object's position and velocity throughout given such time
3. Allow users to modify selected parameters in such a way that would make observations of the object's motion and velocity
4. Provide a graphical presentation of the simulated orbital path
5. Incorporate game-challenges in-between challenges to make the game more interesting.

6. Apply computer science principles in developing the system.

1.4 Significance of the Project

The project ORBIT can help students and learners have an interactive way to explore orbital motion by allowing them to manipulate variables that are within their control and observe their effects on the movement and trajectory of an object. This can help users better understand the relationship between different orbital variables by allowing them to experiment with different conditions and see the resulting changes in real time. The project can also provide a more engaging approach to learning orbital motion by incorporating interactive and game-based elements.

1.5 Scope and Limitations

The project will focus on developing a Python-based interactive simulation game that models orbital motion and allows users to experiment with selected variables affecting an object's trajectory. The simulation will primarily focus on the effects of mass, initial velocity, and distance on orbital motion. Users will be able to modify these variables, run simulations, observe the resulting trajectories, and complete challenges based on specific orbital conditions. The system will also provide graphical representations of the simulated objects and their paths.

The project will use a computational model based on Newtonian gravitational mechanics and numerical calculations to approximate orbital motion. The simulation will be designed to run offline on ordinary school computers and will not require an internet connection for its core functions. The performance of the algorithms and data structures used by the simulation will also be examined as the number of simulated bodies increases.

However, ORBIT will be limited to a simplified representation of orbital systems. It will not attempt to accurately simulate every factor involved in real-world spaceflight, such as atmospheric drag, relativistic effects, radiation pressure, or other complex physical phenomena. The initial version of the project will primarily use a two-dimensional representation of orbital motion and will focus on a limited number of bodies. The accuracy of the simulation will also depend on the numerical methods, time-step size, and computational resources available. The project's evaluation of student understanding will likewise be limited to the participants and orbit-prediction questions used during testing.

Methodology

2.1 System Design and Development

The project will use a development-based approach to create ORBIT, a Python-based interactive simulation game for exploring orbital motion. The system will be designed to accept user-defined values such as mass, initial velocity, and distance, process these values through gravitational calculations, and display the resulting motion of the simulated objects. The development process will include planning the system structure, implementing the simulation algorithms, developing the graphical interface, incorporating game mechanics, and testing the system for functionality and accuracy. The general process of the system will be:

User Input → Parameter Initialization → Gravitational Calculations → Position and Velocity Updates → Trajectory Display → Orbital Outcome → Game Feedback

The system will be designed to operate offline so that its core features can be used on ordinary school computers without requiring an internet connection.

2.2 Orbital Motion Simulation

ORBIT will simulate orbital motion using Newton's law of universal gravitation to calculate the gravitational force between simulated bodies. The calculated force will be used to determine the

acceleration of each body, which will then be used to update its velocity and position over successive time intervals. Numerical integration will be used to approximate the continuous movement of the objects. At each simulation step, the program will calculate the forces acting on the bodies, update their velocities and positions, and record their coordinates to generate their orbital trajectories. The simulation will primarily use a two-dimensional representation to simplify the visualization and computational requirements of the project. The accuracy of the simulation will be evaluated by comparing its results with established orbital relationships, including circular-orbit velocity and predictions based on Kepler's laws. This will allow the developers to determine whether the implemented calculations produce results consistent with known orbital behavior.

2.3 Algorithms, Data Structures, and Game Mechanics

The project will apply algorithms for calculating gravitational interactions, updating the position and velocity of simulated bodies, determining orbital outcomes, and managing game challenges. Data structures will be used to store information about each body, including its mass, position, velocity, and recorded trajectory. Object-oriented programming will also be applied to organize the properties and behaviors of simulated objects. ORBIT will incorporate interactive game mechanics in which users can modify selected orbital parameters, predict the resulting behavior, and then run the simulation to compare their predictions with the actual outcome. Challenges will be designed around different orbital conditions, encouraging users to experiment with mass, velocity, and distance. Unlike simulations that only demonstrate predetermined orbital paths, ORBIT will combine orbital calculations with user-controlled variables, prediction-based challenges, and game mechanics. This allows users to actively test different conditions rather than only observe a preconfigured simulation. The performance of the implemented algorithms and data structures will also be examined as the number of simulated bodies increases.

2.4 Tools and Technologies

The project will use Python as its primary programming language because of its support for mathematical calculations, object-oriented programming, data structures, and rapid development. Pygame will be used to create the graphical interface, display simulated objects and trajectories, process user input, and implement the interactive game elements. Python's built-in mathematical functions and data structures will be used for calculations and storage of simulation data. Object-oriented programming will be used to represent simulated bodies as objects with properties such as mass, position, and velocity, allowing the system to manage multiple bodies in an organized manner. The selected tools will allow the project to combine physics-based calculations with graphical interaction and game mechanics while keeping the system suitable for development and use on ordinary school computers.